AS PER LATEST SYLLABUS

MCA-201

Java Technologies

LEARN • BUILD • INNOVATE • EXCEL

By Virendra Goura


```
public class HelloJava {
    public static void main(String[] args) {
        System.out.println("Welcome to
        Java Technologies!");
    }
}
// Code. Compile. Run. Succeed.
```

EASY TO UNDERSTAND | PRACTICAL EXAMPLES | CONCEPTS MADE SIMPLE | BOOST YOUR CODING SKILLS

www.virendragoura.com

PREFACE

Dear Reader,

It is a privilege to present this comprehensive collection of RTU MCA Semester Examination Notes, designed to cover the complete syllabus with clarity and academic accuracy. Every concept, definition, explanation, and example has been organized to support thorough understanding and effective exam preparation.

The purpose of these notes is to simplify complex topics and provide a reliable study resource that can assist in both detailed learning and quick revision. Continuous effort has been made to ensure correctness and relevance; however, learning grows when readers engage, question, and explore further.

May these notes serve as a strong academic foundation and contribute meaningfully to your preparation and future growth.

Warm regards,
Virendra Goura
www.virendragoura.com

DISCLAIMER

This e-book has been created with utmost care, sincere effort, and extensive proofreading. However, despite all attempts to avoid mistakes, there may still be some errors, omissions, or inaccuracies that remain unnoticed.
This e-book is issued with the understanding that neither the author nor the publisher shall be held responsible for any loss, damage, or misunderstanding arising from the use of the information contained within.
All content provided is for educational and informational purposes only.

MCA - 201 Java Technologies

Unit-1 Introduction to Java

OOP in Java, Characteristics of Java, Fundamental Programming Structures in Java, Abstract Class, Interfaces, Defining Methods, Inheritance, Overloading, Overriding, Packages, Exception Handling, Threads, Thread Life-Cycle.

Unit-2 J2EE Overview

Need of J2EE, J2EE Architecture, J2EE APIs, J2EE Containers. Web Application Basics, Architecture and Challenges of Web Application, Servlet Life Cycle, Developing and Deploying Servlets, Exploring Deployment Descriptor (web.xml), Handling Request and Response, Initializing a Servlet. Servlet Chaining, Session Tracking and Management.

Unit-3 JDBC

The JDBC Connectivity Model, Types of JDBC Drivers, Basic steps to JDBC, setting up a connection to database, Creating and executing SQL statements, ResultSet and ResultSet Metadata Object, Accessing Database.

Unit-4 Java Server Pages

Basic JSP Architecture, Life Cycle of JSP, JSP Tags & Expressions, JSP Implicit Objects, JSP Directives, Tag Libraries, Using JDBC with JSP, Accessing a Database, Adding a Form, Updating the Database.

Unit-5 Introduction to Spring

Overview of Spring Framework- Inversion of Control / Dependency Injection Concepts, Aspect Oriented Programming - concept, Spring MVC Architecture, Bean Factory and Application Context, Attaching and Populating beans, Injecting data through setters and constructors, Listening on events, Publishing events, Spring MVC Layering, Dispatcher Servlet, Writing a Controller, DAO, Models, Services, Spring Configuration File, Error handling Strategy.

Unit-1 Introduction to Java

OOP in Java, Characteristics of Java, Fundamental Programming Structures in Java, Abstract Class, Interfaces, Defining Methods, Inheritance, Overloading, Overriding, Packages, Exception Handling, Threads, Thread Life-Cycle.

Introduction

Java is one of the most popular, secure, and powerful programming languages used for software development. Java was developed by James Gosling at Sun Microsystems in 1995.

Java is:

- High-level language
- Object-oriented language
- Platform-independent language
- Secure programming language

Java follows the principle:

Write Once, Run Anywhere (WORA)
This means Java programs can run on any operating system that supports Java Virtual Machine (JVM).

Java is widely used in:

- Web applications
- Android development
- Banking software
- Enterprise systems
- Cloud computing
- Scientific applications

Features of Java

1. Simple: Java syntax is easy and similar to C/C++.

2. Object-Oriented

Java supports:

- Classes
- Objects
- Inheritance
- Polymorphism

3. Platform Independent: Java bytecode runs on any operating system using JVM.

4. Secure

Java provides:

- Bytecode verification
- Exception handling
- Automatic memory management

5. Robust: Java provides strong exception handling and garbage collection.

6. Multithreaded: Java supports execution of multiple threads simultaneously.

7. Portable: Java programs can be transferred easily between systems.

8. Dynamic: Java supports dynamic loading of classes during runtime.

OOP in Java

Introduction: Java is an object-oriented programming language. Object-Oriented Programming (OOP) organizes programs using:

- Classes
- Objects

OOP helps in:

- Code reusability
- Data security
- Modular programming
- Easy maintenance

Main Principles of OOP

1. Encapsulation: Encapsulation means wrapping data and methods together into a single unit.

Example:

```
class Student {
    private int marks;
    public void setMarks(int m) {
        marks = m;
    }
    public int getMarks() {
        return marks;
    }
}
```

```
class Test {
    public static void main(String args[]) {
        Student s = new Student();
        s.setMarks(90);
        System.out.println(s.getMarks());
    }
}
```

Output

90

Advantages of Encapsulation

1. Improves security.
2. Prevents unauthorized access.
3. Improves maintainability.

2. Abstraction: Abstraction means hiding implementation details and showing only essential features.

Example:

```
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
class Test {
    public static void main(String args[]) {
        Circle c = new Circle();
        c.draw();
    }
}
```

Output

Drawing Circle

Advantages of Abstraction

1. Reduces complexity.
2. Improves security.

3. Simplifies usage.

3. Inheritance: Inheritance allows one class to acquire properties and methods of another class.

Parent class:

- Base class
- Super class

Child class:

- Derived class
- Sub class

Example of Inheritance

```
class Animal {

    void sound() {

        System.out.println("Animal Sound");
    }
}

class Dog extends Animal {

    void bark() {

        System.out.println("Dog Barks");
    }
}

class Test {

    public static void main(String args[]) {

        Dog d = new Dog();

        d.sound();
        d.bark();
    }
}
```

Output

```
Animal Sound
Dog Barks
```

Advantages of Inheritance

1. Code reusability.
2. Reduces duplication.
3. Simplifies maintenance.

4. Polymorphism: Polymorphism means “many forms”.

Same method behaves differently in different situations.

Example of Polymorphism

```
class A {
    void show() {
        System.out.println("Class A");
    }
}

class B extends A {
    void show() {
        System.out.println("Class B");
    }
}

class Test {
    public static void main(String args[]) {
        B b = new B();
        b.show();
    }
}
```

Output

Class B

Advantages of Polymorphism

1. Improves flexibility.
2. Simplifies coding.
3. Supports method overriding.

Fundamental Programming Structures in Java

Structure of Java Program

Example:

```
class Test {
    public static void main(String args[]) {
        System.out.println("Hello Java");
    }
}
```

Output

Hello Java

Explanation of Java Program

Part	Description
class	Defines class
main()	Starting point
System.out.println()	Displays output

Variables in Java: Variables are memory locations used for storing data.

Example:

int x = 10;

Data Types in Java

Data Type	Example
int	10
float	5.5f
double	25.6
char	'A'
boolean	TRUE

Operators in Java

Operator	Example
Arithmetic	+, -, *, /
Relational	>, <
Logical	&&,
Assignment	=

Control Statements in Java

if Statement

Example:

int x = 10;

if(x > 5) {

```
    System.out.println("Greater");
}
```

Output

Greater

Loops in Java

for Loop

Example:

```
for(int i=1;i<=5;i++) {

    System.out.println(i);
}
```

Output

1
2
3
4
5

Abstract Class in Java: An abstract class is a class that cannot be instantiated directly.

Abstract classes are declared using: abstract keyword.

Features of Abstract Class

1. Contains abstract methods.
2. Contains normal methods.
3. Used for abstraction.

Example of Abstract Class

```
abstract class Vehicle {

    abstract void start();
}

class Car extends Vehicle {

    void start() {

        System.out.println("Car Starts");
    }
}

class Test {

    public static void main(String args[]) {

        Car c = new Car();
```

```
        c.start();
    }
}
```

Output

Car Starts

Interfaces in Java: An interface is a collection of abstract methods.

Interfaces are used for:

- Full abstraction
- Multiple inheritance

Interfaces are declared using: interface keyword.

Example of Interface

```
interface Animal {
    void sound();
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Dog Barks");
    }
}

class Test {
    public static void main(String args[]) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Output

Dog Barks

Advantages of Interfaces

1. Supports abstraction.
2. Supports multiple inheritance.
3. Improves flexibility.

Defining Methods in Java: Methods are blocks of code used for performing specific tasks.

Methods improve:

- Reusability
- Modularity
- Readability

Syntax of Method

```
returnType methodName() {
    statements;
}
```

Example of Method

```
class Test {
    void display() {
        System.out.println("Welcome");
    }
    public static void main(String args[]) {
        Test t = new Test();
        t.display();
    }
}
```

Output

```
Welcome
```

Types of Methods

1. Instance methods
2. Static methods
3. Parameterized methods

Example of Parameterized Method

```
class Test {
    void add(int a,int b) {
        System.out.println(a+b);
    }
    public static void main(String args[]) {
        Test t = new Test();
        t.add(10,20);
    }
}
```

Output

30

Inheritance in Java: Inheritance allows child class to use properties and methods of parent class.

Java uses: extends keyword.

Types of Inheritance

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance

Single Inheritance Example

```
class A {

    void display() {

        System.out.println("Class A");
    }
}

class B extends A {

    void show() {

        System.out.println("Class B");
    }
}

class Test {

    public static void main(String args[]) {

        B b = new B();

        b.display();
        b.show();
    }
}
```

Output

```
Class A
Class B
```

Multilevel Inheritance in Java

```
class Grandparent {
    void showGrandparent() {
        System.out.println("Grandparent Class");
    }
}

class Parent extends Grandparent {
    void showParent() {
```

```
        System.out.println("Parent Class");
    }
}

class Child extends Parent {
    void showChild() {
        System.out.println("Child Class");
    }
}

public class Main {
    public static void main(String[] args) {
        Child obj = new Child();

        obj.showGrandparent();
        obj.showParent();
        obj.showChild();
    }
}
```

Hierarchical Inheritance in Java

```
class Parent {
    void showParent() {
        System.out.println("Parent Class");
    }
}

class Child1 extends Parent {
    void showChild1() {
        System.out.println("Child1 Class");
    }
}

class Child2 extends Parent {
    void showChild2() {
        System.out.println("Child2 Class");
    }
}

public class Main {
    public static void main(String[] args) {

        Child1 obj1 = new Child1();
        Child2 obj2 = new Child2();

        obj1.showParent();
        obj1.showChild1();

        obj2.showParent();
        obj2.showChild2();
    }
}
```

Method Overloading in Java: Method overloading means defining multiple methods with same name but different parameters.

Example of Overloading

```
class Test {
```

```
    void add(int a,int b) {

        System.out.println(a+b);
    }

    void add(int a,int b,int c) {

        System.out.println(a+b+c);
    }

    public static void main(String args[]) {

        Test t = new Test();

        t.add(10,20);
        t.add(10,20,30);
    }
}
```

Output

```
30
60
```

Advantages of Overloading

1. Improves readability.
2. Reduces method name complexity.
3. Supports compile-time polymorphism.

Method Overriding in Java: Method overriding means redefining parent class method in child class.

Example of Overriding

```
class A {

    void show() {

        System.out.println("Parent Class");
    }
}

class B extends A {

    void show() {

        System.out.println("Child Class");
    }
}
class Test {

    public static void main(String args[]) {

        B b = new B();

        b.show();
```

```
    }
}
```

Output

Child Class

Advantages of Overriding

1. Supports runtime polymorphism.
2. Improves flexibility.
3. Allows specialized implementation.

Packages in Java

A package is a collection of related classes and interfaces.

Packages help:

- Organize code
- Prevent naming conflicts
- Improve modularity

Creating Package

Example:

```
package mypack;
```

Importing Package

Example:

```
import java.util.*;
```

Types of Packages

1. Built-in packages
2. User-defined packages

Advantages of Packages

1. Better organization.
2. Access protection.
3. Avoids naming conflicts.

Exception Handling in Java

Exception handling manages runtime errors and prevents abnormal program termination.

Java provides:

- try
- catch
- finally
- throw

Example of Exception Handling

```
class Test {
    public static void main(String args[]) {
        try {
            int x = 10/0;
        }
        catch(Exception e) {
            System.out.println("Error Occurred");
        }
    }
}
```

Output

```
Error Occurred
```

finally Block

The finally block always executes.

Example:

```
class Test {
    public static void main(String args[]) {
        try {
            int x = 10/2;
            System.out.println(x);
        }
        finally {
            System.out.println("Program Ended");
        }
    }
}
```

Output

```
5
Program Ended
```

Advantages of Exception Handling

1. Prevents program crash.
2. Improves reliability.
3. Simplifies debugging.

Threads in Java: A thread is the smallest unit of execution.

Multithreading allows multiple tasks to run simultaneously.

Advantages of Threads

1. Faster execution.
2. Better CPU utilization.
3. Improved performance.

Creating Thread Using Thread Class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread Running");
    }
    public static void main(String args[]) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Output

Thread Running

Creating Thread Using Runnable Interface

```
class MyThread implements Runnable {
    public void run() {
        System.out.println("Runnable Thread");
    }
    public static void main(String args[]) {
        MyThread m = new MyThread();
        Thread t = new Thread(m);
        t.start();
    }
}
```

Output

Runnable Thread

Thread Life-Cycle in Java: A thread passes through different states during execution.

States of Thread Life-Cycle

State	Description
New	Thread created
Runnable	Ready to run
Running	Executing
Waiting	Waiting for resource
Dead	Execution completed

Thread Life-Cycle Diagram

New → Runnable → Running → Waiting → Dead

Important Thread Methods

Method	Purpose
start()	Starts thread
run()	Executes thread
sleep()	Pauses thread
join()	Waits for completion

Example of sleep() Method

```
class Test extends Thread {
    public void run() {
        try {
            for(int i=1;i<=3;i++) {
                System.out.println(i);
                Thread.sleep(1000);
            }
        }
        catch(Exception e) {
            System.out.println(e);
        }
    }
    public static void main(String args[]) {
        Test t = new Test();
        t.start();
    }
}
```

Output

```
1
2
3
```

Unit-2 J2EE Overview

Need of J2EE, J2EE Architecture, J2EE APIs, J2EE Containers. Web Application Basics, Architecture and Challenges of Web Application, Servlet Life Cycle, Developing and Deploying Servlets, Exploring Deployment Descriptor (web.xml), Handling Request and Response, Initializing a Servlet. Servlet Chaining, Session Tracking and Management.

Introduction to J2EE

J2EE stands for: Java 2 Enterprise Edition

J2EE is a platform developed for creating large-scale, distributed, multi-tier, and enterprise-level applications using Java technology. It extends the capabilities of core Java by providing a complete environment for developing server-side and web-based applications.

Modern business applications require:

- Security
- Scalability
- Reliability
- Distributed computing
- Database connectivity
- Web support
- Transaction management

Core Java alone is not sufficient for handling such enterprise requirements. Therefore, J2EE was introduced to simplify enterprise application development.

J2EE provides:

- APIs
- Frameworks
- Runtime environments
- Web technologies
- Enterprise services

J2EE applications are widely used in:

- Banking systems
- E-commerce websites
- Airline reservation systems
- Hospital management systems
- Government portals
- ERP systems
- Enterprise business applications

Evolution of Java Enterprise Technology

Java technology evolved in different stages:

Version	Purpose
J2SE	Standard desktop applications
J2EE	Enterprise applications
J2ME	Mobile applications

Later, J2EE became:

Java EE
and currently it is known as:

Jakarta EE

Need of J2EE

Large organizations require applications that can:

- Handle thousands of users
- Process huge databases
- Support distributed computing
- Provide secure communication
- Maintain transactions

Traditional applications face many limitations such as:

- Lack of scalability
- Security issues
- Difficult maintenance
- Limited web support

J2EE was developed to overcome these problems.

Why J2EE is Required

1. Enterprise Application Development: J2EE supports development of enterprise-level applications that handle:

- Large databases
- Thousands of transactions
- Multiple users simultaneously

2. Platform Independence: J2EE applications run on different operating systems because Java bytecode executes on JVM.

3. Distributed Computing Support: J2EE supports communication between applications running on different machines over networks.

4. Web Application Development: J2EE provides technologies such as:

- Servlets
- JSP
- EJB

for dynamic web applications.

5. Database Connectivity: J2EE supports database interaction using JDBC.

Applications can connect with:

- MySQL
- Oracle
- PostgreSQL

6. Security Support

J2EE provides:

- Authentication
- Authorization
- Secure communication

7. Transaction Management: J2EE handles transactions automatically in enterprise systems.

Example:

- Banking transaction processing

8. Reusability and Modularity: J2EE promotes reusable software components.

Advantages of J2EE

1. Platform independent architecture.
2. Simplifies enterprise application development.
3. Supports distributed computing.
4. Provides scalability and security.
5. Supports web technologies.
6. Provides transaction management.

7. Supports component reusability.
8. Simplifies database interaction.

J2EE Architecture

J2EE uses multi-tier architecture for developing enterprise applications.

Multi-tier architecture divides application into separate layers. Each layer performs specific tasks independently.

This architecture improves:

- Maintainability
- Scalability
- Security
- Performance

Layers of J2EE Architecture

J2EE architecture mainly contains four tiers:

1. Client Tier
2. Web Tier
3. Business Tier
4. Enterprise Information System Tier

1. Client Tier: The client tier is the front-end layer that interacts directly with users.

Users send requests through: Web browsers, Mobile applications, Desktop applications

Functions of Client Tier

1. User interaction.
2. Sending requests to server.
3. Displaying responses to users.

Examples of Client Tier

- Google Chrome
- Mozilla Firefox
- Android applications

2. Web Tier: The web tier processes client requests and generates responses.

Technologies used: Servlets, JSP

Functions of Web Tier

1. Request handling.
2. Response generation.
3. Session management.
4. User authentication.

Example of Web Tier

When user submits login form:

- Web tier receives request
- Validates information
- Sends data to business tier

3. Business Tier: Business tier contains application business logic.

Technologies used: EJB, JavaBeans

Functions of Business Tier

1. Business rule processing.
2. Transaction management.
3. Data validation.
4. Communication with database tier.

Example of Business Logic

Banking system:

- Check balance
- Transfer money
- Calculate interest

4. Enterprise Information System Tier: This tier manages enterprise resources such as databases.

Examples: MySQL, Oracle, ERP systems

Functions of EIS Tier

1. Data storage.
2. Data retrieval.
3. Database management.
4. Persistent storage handling.

J2EE Architecture Diagram

Client Tier
↓
Web Tier
↓
Business Tier
↓
Database Tier

Advantages of Multi-Tier Architecture

1. Better scalability.
2. Improved security.
3. Easier maintenance.
4. Better resource management.
5. Modular application design.

J2EE APIs: J2EE provides various APIs for enterprise application development.

API means: Application Programming Interface

APIs provide predefined classes and methods that simplify programming.

Important J2EE APIs

API	Purpose
JDBC	Database connectivity
Servlet API	Web applications
JSP	Dynamic web pages
EJB	Enterprise business components
JMS	Messaging services
JNDI	Naming and directory services
RMI	Remote method invocation

JDBC API

JDBC stands for: Java Database Connectivity

Used for: Database connection, SQL query execution, Data retrieval

Example of JDBC

Connection con;

Servlet API

Used for:

- Handling HTTP requests

- Generating dynamic responses

JSP API

JSP stands for: Java Server Pages

Used for creating dynamic web pages.

EJB API

EJB stands for: Enterprise Java Beans

Used for enterprise business logic.

JMS API

JMS stands for: Java Message Service

Used for message communication between applications.

J2EE Containers

Containers provide runtime environment for J2EE components.

Containers manage:

- Lifecycle
- Security
- Transactions
- Resource allocation

Responsibilities of Containers

1. Object creation.
2. Memory management.
3. Security handling.
4. Transaction management.
5. Request processing.

Types of J2EE Containers

Container	Purpose
Applet Container	Executes applets
Application Client Container	Executes desktop applications

Web Container	Executes servlets and JSP
EJB Container	Executes enterprise beans

Web Container

Web container manages:

- Servlets
- JSP
- Session tracking

Examples:

- Apache Tomcat
- GlassFish

Responsibilities of Web Container

1. Servlet lifecycle management.
2. Request-response handling.
3. Session management.
4. Security management.

EJB Container: EJB container manages enterprise beans and business logic.

Responsibilities of EJB Container

1. Transaction management.
2. Security services.
3. Persistence handling.
4. Remote communication.

Web Application Basics

A web application is software accessed through web browser using network connection.

Examples:

- Gmail
- Facebook
- Amazon

Components of Web Application

1. Client browser

2. Web server
3. Application server
4. Database server

Working of Web Application

1. User sends request through browser.
2. Web server receives request.
3. Application server processes request.
4. Database operations occur.
5. Response sent back to browser

Architecture of Web Application

Web applications mainly use: Client-Server Architecture

Web Application Architecture Diagram

Browser
↓
Web Server
↓
Application Server
↓
Database

Challenges of Web Applications

1. Security Issues: Web applications are vulnerable to:

- Hacking
- SQL injection
- Session hijacking

2. Scalability Problems: Applications must handle large number of users.

3. Session Management: HTTP protocol is stateless.

Managing user sessions becomes difficult.

4. Performance Issues: Heavy traffic reduces application speed.

5. Database Management: Large databases become difficult to maintain.

Introduction to Servlet

A servlet is a Java program that runs on server side and handles client requests.

Servlets are used for:

- Dynamic web applications
- Request processing
- Response generation

Servlets are part of: Java Servlet API

Advantages of Servlets

1. Platform independent.
2. Better performance than CGI.
3. Supports multithreading.
4. Secure processing.
5. Reusable components.

Servlet Life Cycle : Servlet lifecycle defines stages through which servlet passes during execution.

Stages of Servlet Lifecycle

1. Loading
2. Initialization
3. Service
4. Destruction

1. Loading Phase: Servlet class is loaded into memory by web container.

2. Initialization Phase: init() method executes once.

Used for:

- Resource allocation
- Initialization tasks

3. Service Phase: service() method handles client requests.

Depending on request type:

- doGet()
- doPost()

methods execute.

4. Destruction Phase: destroy() method executes before servlet removal.

Used for:

- Resource cleanup
- Connection closing

Servlet Lifecycle Diagram

Loading
↓
init()
↓
service()
↓
destroy()

Example of Servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class FirstServlet
extends HttpServlet {

    public void doGet(
    HttpServletRequest req,
    HttpServletResponse res)
    throws IOException {

        PrintWriter out =
        res.getWriter();

        out.println("Hello Servlet");
    }
}
```

Output

Hello Servlet

Developing and Deploying Servlets

Steps for Developing Servlet

1. Create servlet class.
2. Compile servlet.
3. Configure web.xml.
4. Deploy on server.
5. Execute in browser.

Servlet Directory Structure

```
WebApp
 ├── WEB-INF
 │     ├── web.xml
 │     └── classes
```

Exploring Deployment Descriptor (web.xml)

web.xml is deployment descriptor used for servlet configuration.

It defines:

- Servlet mapping
- Welcome pages
- Initialization parameters

Example of web.xml

```
<web-app>

<servlet>
   <servlet-name>First</servlet-name>

   <servlet-class>
      FirstServlet
   </servlet-class>
</servlet>

<servlet-mapping>
   <servlet-name>First</servlet-name>

   <url-pattern>/hello</url-pattern>
</servlet-mapping>

</web-app>
```

Handling Request and Response

Servlets use:

- HttpServletRequest
- HttpServletResponse

HttpServletRequest

Used for:

- Reading form data
- Getting parameters
- Reading client information

Example

```
String name =
request.getParameter("name");
```

HttpServletResponse

Used for:

- Sending output
- Generating responses

Example

```
PrintWriter out =
response.getWriter();

out.println("Welcome");
```

Initializing a Servlet

Servlet initialization is performed using:

```
init()
```
method.

Example of init()

```
public void init() {

    System.out.println(
    "Servlet Initialized");
}
```

Servlet Chaining

Servlet chaining means passing request from one servlet to another servlet.

Used for:

- Modular processing
- Request forwarding

RequestDispatcher Interface

Methods:

- forward()
- include()

Example of Servlet Chaining

```
RequestDispatcher rd =
request.getRequestDispatcher(
"Second");

rd.forward(request,response);
```

Session Tracking and Management

Session tracking maintains user data across multiple requests.

HTTP is stateless protocol, therefore session management is necessary.

Need of Session Tracking

Without session tracking:

- User login information is lost.
- Shopping cart data disappears.

Techniques of Session Tracking

1. Cookies
2. URL Rewriting
3. Hidden Form Fields
4. HttpSession

Cookies: Cookies store small data on client browser.

Example:

```
Cookie c =
new Cookie("user","Veer");

response.addCookie(c);
```

URL Rewriting: Session information added to URL.

Example:

```
login.jsp?user=Veer
```

Hidden Form Fields: Stores hidden information inside forms.

Example:

```
<input type="hidden"
name="user"
value="Veer">
```

HttpSession

Stores session data on server side.

Example:

```
HttpSession session =
request.getSession();

session.setAttribute(
"user","Veer");
```

Advantages of Session Management

1. Maintains user state.
2. Improves user experience.
3. Supports authentication.
4. Essential for e-commerce applications.

Advantages of Servlets

1. Faster than CGI.
2. Platform independent.
3. Supports multithreading.
4. Better security.
5. Efficient resource utilization.

Applications of J2EE

J2EE is used in:

- Banking systems
- E-commerce websites
- Hospital management systems
- ERP applications
- Government portals
- Airline reservation systems

Unit-3 JDBC

The JDBC Connectivity Model, Types of JDBC Drivers, Basic steps to JDBC, setting up a connection to database, Creating and executing SQL statements, ResultSet and ResultSet Metadata Object, Accessing Database.

Introduction to JDBC

JDBC stands for: Java Database Connectivity

JDBC is a standard Java API used for establishing communication between Java applications and relational databases. JDBC allows Java programs to connect with databases and perform operations such as:

- Storing data
- Retrieving data
- Updating records
- Deleting records
- Managing transactions

JDBC acts as a bridge between:

- Java application
- Database Management System (DBMS)

Using JDBC, Java applications can interact with different databases such as:

- MySQL
- Oracle
- PostgreSQL
- SQLite
- Microsoft SQL Server

JDBC is one of the most important technologies used in:

- Enterprise applications
- Web applications
- Banking systems
- E-commerce applications
- ERP systems

Need of JDBC

Modern software applications generate and process huge amounts of data daily. This data must be stored securely and efficiently inside databases.

Core Java alone cannot directly communicate with databases. Therefore, JDBC was introduced to provide standard database connectivity.

Without JDBC:

- Java applications cannot execute SQL queries.
- Data cannot be stored permanently.
- Database interaction becomes difficult.

JDBC solves these problems by providing:

- Standard database communication
- SQL query execution
- Cross-platform database support

Why JDBC is Important

1. Database Connectivity: JDBC allows Java programs to connect with databases easily.

2. Data Management

Applications can:

- Insert records
- Retrieve records
- Modify records
- Delete records

3. Enterprise Application Support: Large enterprise systems use JDBC for handling database operations.

4. Platform Independence: JDBC works on multiple operating systems.

5. SQL Support: JDBC allows execution of SQL queries directly from Java applications.

Advantages of JDBC

1. Platform independent connectivity.
2. Supports multiple databases.
3. Easy integration with Java applications.
4. Supports SQL query execution.
5. Provides secure database interaction.
6. Simplifies enterprise application development.
7. Supports transaction management.
8. Improves application scalability.

Applications of JDBC

JDBC is widely used in:

- Banking systems
- Online shopping websites
- Hospital management systems
- Airline reservation systems
- Student management systems
- Enterprise web applications

JDBC Architecture

JDBC architecture defines the interaction between:

- Java application
- JDBC API
- JDBC driver
- Database

The architecture provides standardized database communication.

Components of JDBC Architecture

1. Java Application
2. JDBC API
3. DriverManager
4. JDBC Driver
5. Database

JDBC Architecture Diagram

Java Application
↓
JDBC API
↓
DriverManager
↓
JDBC Driver
↓
Database

Working of JDBC Architecture

1. Java application sends request to JDBC API.

2. JDBC API forwards request to DriverManager.
3. DriverManager selects appropriate JDBC driver.
4. JDBC driver communicates with database.
5. Database processes query and returns result.
6. Result is sent back to Java application.

JDBC Connectivity Model

The JDBC connectivity model defines how Java applications establish communication with relational databases using JDBC API and JDBC drivers.

The model includes:

- JDBC API
- JDBC drivers
- DriverManager
- Database connection objects

Main Components of JDBC Connectivity Model

Component	Purpose
JDBC API	Provides interfaces and classes
DriverManager	Manages database drivers
JDBC Driver	Communicates with database
Connection	Represents database connection
Statement	Executes SQL queries
ResultSet	Stores query results

JDBC API: JDBC API provides interfaces and classes for database programming.

Important interfaces:

- Connection
- Statement
- PreparedStatement
- ResultSet

DriverManager

DriverManager is responsible for:

- Loading database drivers

- Managing database connections

Example:

DriverManager.getConnection()

Connection Object: Connection object represents active connection between Java application and database.

Example:

Connection con;

Statement Object: Statement object is used for executing SQL queries.

Example:

Statement stmt;

ResultSet Object: ResultSet stores records returned from database query.

Types of JDBC Drivers

JDBC drivers are software components that convert JDBC calls into database-specific communication.

There are four types of JDBC drivers.

Type–1 Driver : JDBC-ODBC Bridge Driver: This driver converts JDBC calls into ODBC calls.

Architecture

Java Application
↓
JDBC Driver
↓
ODBC Driver
↓
Database

Advantages

1. Easy to use.
2. Suitable for learning purposes.

Disadvantages

1. Slow performance.
2. Platform dependent.
3. Requires ODBC installation.
4. Not suitable for enterprise applications.

Type–2 Driver : Native API Driver: This driver converts JDBC calls into native database API calls.

Advantages

1. Better performance than Type–1.
2. Faster database communication.

Disadvantages

1. Requires native libraries.
2. Platform dependent.
3. Difficult deployment.

Type–3 Driver : Network Protocol Driver: This driver converts JDBC calls into network protocol calls using middleware server.

Advantages

1. Platform independent.
2. Suitable for distributed systems.
3. Supports multiple databases.

Disadvantages

1. Requires middleware server.
2. Slower than Type–4 driver.

Type–4 Driver : Thin Driver: This driver directly communicates with database using native protocol.

It is the most widely used JDBC driver.

Advantages

1. High performance.
2. Platform independent.
3. No middleware required.
4. Fast communication.
5. Suitable for enterprise applications.

Disadvantages

1. Database-specific driver required.

Comparison of JDBC Drivers

Driver Type	Platform Independent	Performance

Type–1	No	Slow
Type–2	No	Medium
Type–3	Yes	Good
Type–4	Yes	Best

Basic Steps to JDBC

Database programming using JDBC follows several standard steps.

Steps in JDBC Programming

1. Import JDBC package
2. Load JDBC driver
3. Establish database connection
4. Create statement object
5. Execute SQL query
6. Process results
7. Close database connection

Step–1 : Import JDBC Package: Required package:

import java.sql.*;
This package contains:

- Connection
- Statement
- ResultSet
- DriverManager

Step–2 : Load JDBC Driver: Driver loading registers database driver with DriverManager.

Example

Class.forName(

"com.mysql.cj.jdbc.Driver");

Explanation

- Class.forName() loads JDBC driver into memory.
- MySQL driver is loaded dynamically.

Step–3 : Establish Database Connection

Connection establishes communication with database.

Syntax

```
Connection con =
DriverManager.getConnection(
url,username,password);
```

Example

```
Connection con =
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test",
"root",
"root");
```

Output

Database Connected

Explanation of Database URL

jdbc:mysql://localhost:3306/test

Part	Meaning
jdbc	JDBC protocol
mysql	Database type
localhost	Server address
3306	Port number
test	Database name

Step–4 : Create Statement Object: Statement object executes SQL queries.

Example

```
Statement stmt =
con.createStatement();
```

Types of Statement Objects

1. Statement
2. PreparedStatement
3. CallableStatement

Statement Interface

Used for normal SQL queries.

Example:

```
Statement stmt;
```

PreparedStatement Interface: Used for parameterized queries.

Advantages:

1. Faster execution.
2. Prevents SQL injection.
3. Reusable queries.

CallableStatement Interface: Used for stored procedures.

Step–5 : Execute SQL Queries

executeQuery()

Used for:

- SELECT queries

Example:

```
ResultSet rs =
stmt.executeQuery(
"SELECT * FROM student");
```

executeUpdate()

Used for:

- INSERT
- UPDATE
- DELETE

Example:

```
stmt.executeUpdate(
"INSERT INTO student VALUES(1,'Veer')");
```

Step–6 : Process Results: Query results are processed using ResultSet object.

Example

```
while(rs.next()) {

   System.out.println(
   rs.getInt(1));

   System.out.println(
   rs.getString(2));
}
```

Output

```
1
Veer
```

Step–7 : Close Connection: Database resources must be closed after use.

Example con.close();

Complete JDBC Program

```
import java.sql.*;

class Test {

    public static void main(String args[]) {

        try {

            Class.forName(
            "com.mysql.cj.jdbc.Driver");

            Connection con =
            DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test",
            "root",
            "root");

            Statement stmt =
            con.createStatement();

            ResultSet rs =
            stmt.executeQuery(
            "SELECT * FROM student");

            while(rs.next()) {

                System.out.println(
                rs.getInt(1)
                +" "
                +rs.getString(2));
            }

            con.close();
        }

        catch(Exception e) {

            System.out.println(e);
        }
    }
}
```

Output

```
1 Veer
2 Ram
```

Setting Up a Connection to Database

Setting up connection is one of the most important tasks in JDBC.

Connection is established using:

DriverManager.getConnection()

Requirements for Database Connection

1. JDBC Driver

2. Database URL
3. Username
4. Password

Example of Database Connection

```
Connection con =
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/college",
"root",
"root");
```

Advantages of Database Connection

1. Enables database communication.
2. Supports SQL execution.
3. Allows data retrieval and storage.

Creating and Executing SQL Statements

SQL statements are executed using Statement or PreparedStatement objects.

Types of SQL Commands

Command	Purpose
CREATE	Create table
INSERT	Insert records
SELECT	Retrieve records
UPDATE	Modify records
DELETE	Delete records

Creating Table Example

```
stmt.executeUpdate(
"CREATE TABLE student(id INT,name VARCHAR(20))");
```

Inserting Records Example

```
stmt.executeUpdate(
"INSERT INTO student VALUES(1,'Veer')");
```

Updating Records Example

```
stmt.executeUpdate(
"UPDATE student SET name='Ram' WHERE id=1");
```

Deleting Records Example

```
stmt.executeUpdate(
"DELETE FROM student WHERE id=1");
```

ResultSet Object

ResultSet stores records returned from SELECT query.

ResultSet behaves like virtual table containing rows and columns.

Features of ResultSet

1. Stores query results.
2. Supports row-by-row processing.
3. Provides navigation methods.
4. Retrieves column values.

Important Methods of ResultSet

Method	Purpose
next()	Moves to next row
getInt()	Retrieves integer value
getString()	Retrieves string value
getDouble()	Retrieves double value

Example of ResultSet

```
while(rs.next()) {

    System.out.println(
    rs.getInt(1));

    System.out.println(
    rs.getString(2));
}
```

Output

```
1
Veer
```

ResultSet Metadata Object

ResultSetMetaData provides information about ResultSet structure.

It gives:

- Number of columns
- Column names
- Column data types

Creating ResultSetMetaData Object

```
ResultSetMetaData rsmd =
rs.getMetaData();
```

Important Methods of ResultSetMetaData

Method	Purpose
getColumnCount()	Returns total columns
getColumnName()	Returns column name
getColumnTypeName()	Returns column type

Example of ResultSetMetaData

```
ResultSetMetaData rsmd =
rs.getMetaData();

System.out.println(
rsmd.getColumnCount());

System.out.println(
rsmd.getColumnName(1));
```

Output

```
2
id
```

Accessing Database Using JDBC

JDBC allows complete database access through Java programs.

Operations include:

- Creating databases
- Creating tables
- Inserting records
- Retrieving records
- Updating records
- Deleting records

Example of Database Access

```
import java.sql.*;

class Test {

    public static void main(String args[]) {

        try {

            Connection con =
            DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test",
            "root",
            "root");
```

```
            Statement stmt =
            con.createStatement();

            stmt.executeUpdate(
            "INSERT INTO student VALUES(1,'Veer')");

            ResultSet rs =
            stmt.executeQuery(
            "SELECT * FROM student");

            while(rs.next()) {

               System.out.println(
               rs.getInt(1)
               +" "
               +rs.getString(2));
            }

            con.close();
         }

         catch(Exception e) {

            System.out.println(e);
         }
      }
}
```

Output

1 Veer

PreparedStatement in JDBC

PreparedStatement is precompiled SQL statement.

It improves:

- Performance
- Security
- Efficiency

Advantages of PreparedStatement

1. Faster execution.
2. Prevents SQL injection attacks.
3. Reusable queries.
4. Better performance.

Example of PreparedStatement

```
PreparedStatement ps =
con.prepareStatement(
"INSERT INTO student VALUES(?,?)");

ps.setInt(1,1);
```

ps.setString(2,"Veer");

ps.executeUpdate();

Transaction Management in JDBC

Transaction is group of SQL operations executed together.

JDBC Transaction Methods

Method	Purpose
commit()	Saves changes
rollback()	Cancels changes

Example of commit()

```
con.commit();
```

Example of rollback()

```
con.rollback();
```

Exception Handling in JDBC

JDBC operations may generate exceptions such as:

- SQLException
- ClassNotFoundException

Example:

```
catch(SQLException e) {
    System.out.println(e);
}
```

Advantages of JDBC

1. Standard database connectivity.
2. Supports multiple databases.
3. Platform independent.
4. Secure database interaction.
5. Supports enterprise applications.
6. Easy SQL integration.
7. Efficient transaction management.

Limitations of JDBC

1. Large amount of coding required.

2. Manual resource management.
3. Database-specific drivers required.
4. Complex error handling.

Applications of JDBC

JDBC is widely used in:

- Banking systems
- E-commerce websites
- Enterprise software
- Online examination systems
- Hospital management systems
- Web applications

Unit-4 Java Server Pages

Basic JSP Architecture, Life Cycle of JSP, JSP Tags & Expressions, JSP Implicit Objects, JSP Directives, Tag Libraries, Using JDBC with JSP, Accessing a Database, Adding a Form, Updating the Database.

Introduction to JSP

JSP stands for: Java Server Pages

JSP is a server-side technology used for creating dynamic web pages using Java. JSP allows developers to embed Java code inside HTML pages to generate dynamic content.

JSP is an important part of:

J2EE (Java 2 Enterprise Edition)
JSP simplifies web application development because it combines:

- HTML
- Java
- JSP tags
- Expression Language

JSP pages are executed on server side and converted into servlets by the web container.

Need of JSP

Static HTML pages cannot generate dynamic content. Modern web applications require:

- Dynamic pages
- User interaction
- Database connectivity
- Form processing
- Session management

Servlets can generate dynamic content, but writing HTML inside servlets becomes difficult and complex.

JSP was introduced to solve these problems.

Why JSP is Used

1. Dynamic Web Pages: JSP generates dynamic content at runtime.

2. Simplifies Web Development: HTML and Java can be combined easily.

3. Better Separation of Logic and Design: Designers work on HTML while programmers work on Java code.

4. Database Interaction: JSP supports JDBC for database access.

5. Session Handling: JSP supports session management.

Advantages of JSP

1. Easy development of web applications.
2. Platform independent.
3. Supports reusable components.
4. Better code organization.
5. Supports Java APIs.
6. Simplifies dynamic page creation.
7. Supports database connectivity.

JSP Architecture

JSP architecture defines how JSP pages are processed by web server and web container.

When a client requests a JSP page:

1. Request goes to web server.
2. Web server forwards request to JSP container.
3. JSP page converts into servlet.
4. Servlet is compiled and executed.
5. Response sent back to browser.

JSP Architecture Diagram

Browser
↓
Web Server
↓
JSP Container
↓
Servlet
↓
Database

Components of JSP Architecture

Component	Purpose
Browser	Sends request
Web Server	Handles HTTP requests
JSP Container	Converts JSP into servlet
Servlet Engine	Executes servlet
Database	Stores data

Working of JSP Architecture

1. Browser requests JSP page.
2. JSP container checks JSP file.
3. JSP page converts into servlet.
4. Servlet compiles into bytecode.
5. Servlet executes.
6. Dynamic response generated.
7. Response sent to browser.

JSP Life Cycle

JSP lifecycle defines stages through which JSP page passes during execution.

The JSP lifecycle is managed by:

- JSP container
- Web container

Stages of JSP Life Cycle

1. Translation Phase
2. Compilation Phase
3. Class Loading Phase
4. Instantiation Phase
5. Initialization Phase
6. Request Processing Phase
7. Destruction Phase

1. Translation Phase: JSP page converts into servlet source code.

2. Compilation Phase: Generated servlet source code compiles into bytecode.

3. Class Loading Phase: Servlet class loads into memory.

4. Instantiation Phase: Servlet object is created.

5. Initialization Phase

jspInit() method executes.

Used for:

- Resource initialization

- Database connection setup

6. Request Processing Phase: _jspService() method processes client requests.

7. Destruction Phase: jspDestroy() method executes before JSP removal.

JSP Life Cycle Diagram

Translation
↓
Compilation
↓
Class Loading
↓
Instantiation
↓
jspInit()
↓
_jspService()
↓
jspDestroy()

Example of JSP Page

```
<html>
<body>
<h1>Welcome to JSP</h1>
</body>
</html>
```

Output

Welcome to JSP

JSP Tags and Expressions

JSP provides special tags and expressions for embedding Java code inside HTML pages.

These tags simplify:

- Dynamic content generation
- Java code embedding
- Variable output

Types of JSP Elements

1. Scriptlet Tag
2. Declaration Tag
3. Expression Tag

1. Scriptlet Tag: Scriptlet tag contains Java code executed during request processing.

Syntax:

```
<%
   Java Code
%>
```

Example of Scriptlet Tag

```
<%

int x = 10;
int y = 20;

out.println(x+y);

%>
```

Output

30

2. Declaration Tag: Declaration tag declares variables and methods.

Syntax:

```
<%!

   declaration

%>
```

Example of Declaration Tag

```
<%!

int square(int n) {

   return n*n;
}

%>

<%= square(5) %>
```

Output

25

3. Expression Tag: Expression tag displays output directly on browser.

Syntax:

```
<%= expression %>
```

Example of Expression Tag

```
<%= "Welcome JSP" %>
```

Output

Welcome JSP

JSP Implicit Objects

JSP implicit objects are predefined objects automatically created by JSP container.

Programmers can use them directly without declaration.

Important JSP Implicit Objects

Object	Purpose
request	Handles client request
response	Sends response
out	Displays output
session	Session management
application	Application-level data
config	Servlet configuration
page	Current JSP page
exception	Exception handling

request Object

Used for:

- Reading form data
- Getting parameters

Example:

```
<%

String name =
request.getParameter("name");

out.println(name);

%>
```

response Object

Used for:

- Sending responses
- Redirecting pages

Example:

```
response.sendRedirect("home.jsp");
```

out Object

Used for displaying output.

Example:

out.println("Hello JSP");

session Object

Used for session tracking.

Example:

session.setAttribute(
"user","Veer");

JSP Directives

JSP directives provide instructions to JSP container.

Directives affect entire JSP page.

Types of JSP Directives

1. page Directive
2. include Directive
3. taglib Directive

1. page Directive

Used for:

- Importing packages
- Error handling
- Session control

Syntax:

<%@ page attribute="value" %>

Example

<%@ page import="java.util.*" %>

2. include Directive: Used for including another file.

Example:

<%@ include file="header.jsp" %>

3. taglib Directive: Used for custom tag libraries.

Example:

<%@ taglib uri="mytag" prefix="t" %>

Tag Libraries in JSP

Tag libraries provide custom tags that simplify JSP programming.

Tag libraries reduce Java code inside JSP pages.

Advantages of Tag Libraries

1. Reduces complexity.
2. Improves readability.
3. Reusable components.
4. Better maintainability.

JSTL (JSP Standard Tag Library)

JSTL provides predefined tags for:

- Loops
- Conditions
- Database operations

Example of JSTL

```
<c:out value="Welcome"/>
```

Using JDBC with JSP

JSP pages can interact with databases using JDBC.

JDBC allows JSP applications to:

- Store data
- Retrieve data
- Update records
- Delete records

Steps for JDBC with JSP

1. Import JDBC package
2. Load JDBC driver
3. Establish connection
4. Create statement
5. Execute SQL query
6. Process results
7. Close connection

Example of JDBC with JSP

```
<%@ page import="java.sql.*" %>

<%

Class.forName(
"com.mysql.cj.jdbc.Driver");

Connection con =
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test",
"root",
"root");

Statement stmt =
con.createStatement();

ResultSet rs =
stmt.executeQuery(
"SELECT * FROM student");

while(rs.next()) {

    out.println(
    rs.getInt(1)
    +" "
    +rs.getString(2));
}

con.close();

%>
```

Output

```
1 Veer
2 Ram
```

Accessing a Database

JSP applications access databases using JDBC.

Database operations include:

- Creating tables
- Inserting records
- Retrieving records
- Updating records
- Deleting records

Example of Database Access

```
<%@ page import="java.sql.*" %>

<%

Connection con =
DriverManager.getConnection(
```

```
"jdbc:mysql://localhost:3306/test",
"root",
"root");

Statement stmt =
con.createStatement();

ResultSet rs =
stmt.executeQuery(
"SELECT * FROM student");

while(rs.next()) {

    out.println(
    rs.getString(2));
}

%>
```

Output

```
Veer
Ram
```

Adding a Form in JSP

Forms are used for accepting user input.

JSP forms collect data such as:

- Name
- Password
- Email
- Address

Example of JSP Form

```
<html>

<body>

<form action="save.jsp">

Name:
<input type="text" name="name">

<input type="submit">

</form>

</body>

</html>
```

Output

Form displayed on browser

Reading Form Data

Example:

```
<%

String name =
request.getParameter("name");

out.println(name);

%>
```

Output

Veer

Updating the Database

Database records can be updated using SQL UPDATE statement.

Example of Updating Database

```
<%@ page import="java.sql.*" %>

<%

Connection con =
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test",
"root",
"root");

Statement stmt =
con.createStatement();

stmt.executeUpdate(
"UPDATE student SET name='Ram' WHERE id=1");

out.println("Record Updated");

con.close();

%>
```

Output

Record Updated

Inserting Data into Database Using JSP Form

Example

```
<%@ page import="java.sql.*" %>

<%

String name =
request.getParameter("name");
```

```
Connection con =
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test",
"root",
"root");

PreparedStatement ps =
con.prepareStatement(
"INSERT INTO student(name) VALUES(?)");

ps.setString(1,name);

ps.executeUpdate();

out.println("Data Inserted");

con.close();

%>
```

Output

Data Inserted

Advantages of JSP

1. Easy dynamic web page development.
2. Platform independent.
3. Supports reusable components.
4. Supports JDBC connectivity.
5. Faster development.
6. Better separation of logic and presentation.
7. Supports session management.

Limitations of JSP

1. Mixing Java and HTML increases complexity.
2. Difficult debugging for large applications.
3. Maintenance becomes difficult in complex projects.

Applications of JSP

JSP is widely used in:

- Banking systems
- E-commerce websites
- Hospital management systems
- ERP systems

- Online examination systems
- Enterprise web applications

Difference Between Servlet and JSP

Servlet	JSP
Java code dominant	HTML dominant
Difficult UI design	Easy UI design
Better for business logic	Better for presentation
Requires more coding	Less coding required

Unit-5 Introduction to Spring

Overview of Spring Framework- Inversion of Control / Dependency Injection Concepts, Aspect Oriented Programming - concept, Spring MVC Architecture, Bean Factory and Application Context, Attaching and Populating beans, Injecting data through setters and constructors, Listening on events, Publishing events, Spring MVC Layering, Dispatcher Servlet, Writing a Controller, DAO, Models, Services, Spring Configuration File, Error handling Strategy.

Introduction to Spring Framework

Spring Framework is one of the most popular Java frameworks used for developing enterprise applications. It is an open-source framework developed to simplify Java application development.

Spring provides:

- Lightweight architecture
- Dependency Injection
- Aspect-Oriented Programming
- MVC architecture
- Database support
- Transaction management

Spring is widely used for:

- Enterprise applications
- Web applications
- REST APIs
- Cloud applications
- Microservices

The Spring Framework was developed by Rod Johnson.

Need of Spring Framework

Before Spring, enterprise Java applications using EJB (Enterprise Java Beans) were:

- Complex
- Heavyweight
- Difficult to maintain
- Difficult to test

Spring was introduced to solve these problems by providing:

- Lightweight programming model
- Loose coupling

- Better modularity
- Easier testing
- Simplified development

Features of Spring Framework

1. Lightweight Framework: Spring is lightweight and easy to use.

2. Dependency Injection Support: Spring automatically injects required objects.

3. Aspect Oriented Programming Support: Spring supports modular handling of cross-cutting concerns.

4. MVC Architecture Support: Spring MVC simplifies web application development.

5. Transaction Management: Spring simplifies database transaction handling.

6. Integration Support

Spring integrates easily with:

- Hibernate
- JDBC
- JSP
- Servlet
- REST APIs

Advantages of Spring Framework

1. Simplifies enterprise development.
2. Reduces code complexity.
3. Supports loose coupling.
4. Improves modularity.
5. Easy integration with technologies.
6. Simplifies testing.
7. Provides reusable components.

Overview of Spring Framework: Spring Framework consists of multiple modules.

Important Modules of Spring

Module	Purpose
Core Container	Dependency Injection
Spring MVC	Web application development

JDBC Module	Database connectivity
AOP Module	Aspect-oriented programming
ORM Module	Hibernate integration
Security Module	Authentication and authorization

Spring Framework Architecture

Application
↓
Spring Container
↓
Beans and Services
↓
Database

Inversion of Control (IoC)

Inversion of Control (IoC) is a design principle in which control of object creation and dependency management is transferred from programmer to Spring container.

Normally:

- Programmer creates objects manually.

In Spring:

- Spring container creates and manages objects automatically.

Need of IoC

Without IoC:

- Tight coupling occurs.
- Object management becomes difficult.

IoC solves these problems by:

- Managing objects automatically
- Reducing dependency complexity

Advantages of IoC

1. Reduces tight coupling.
2. Improves modularity.
3. Simplifies testing.
4. Easier maintenance.

Example Without IoC

```
class Engine {

    void start() {
```

```
        System.out.println("Engine Started");
    }
}

class Car {

    Engine e = new Engine();
}
```

Problem

- Car class directly creates Engine object.
- Tight coupling occurs.

Example With IoC

```
class Car {

    Engine e;

    public void setEngine(Engine e) {

        this.e = e;
    }
}
```

Dependency Injection (DI)

Dependency Injection is a technique in which dependencies are provided by Spring container instead of creating them manually.

Dependency means:

- Object required by another object.

Types of Dependency Injection

1. Setter Injection
2. Constructor Injection

Advantages of Dependency Injection

1. Loose coupling.
2. Easier testing.
3. Better code reuse.
4. Simplifies object management.

Setter Injection: Dependencies are injected using setter methods

Example of Setter Injection

```
class Student {

    private String name;
```

```
    public void setName(String name) {

        this.name = name;
    }

    public void display() {

        System.out.println(name);
    }
}
```

XML Configuration

```
<bean id="s"
class="Student">

<property name="name"
value="Veer"/>

</bean>
```

Output

```
Veer
```

Constructor Injection: Dependencies are injected using constructor.

Example of Constructor Injection

```
class Student {

    String name;

    Student(String name) {

        this.name = name;
    }

    void display() {

        System.out.println(name);
    }
}
```

XML Configuration

```
<bean id="s"
class="Student">

<constructor-arg
value="Veer"/>

</bean>
```

Output

```
Veer
```

Aspect Oriented Programming (AOP)

Aspect-Oriented Programming is programming paradigm used for separating cross-cutting concerns from main business logic.

Cross-cutting concerns include:

- Logging
- Security
- Transaction management
- Error handling

Need of AOP

Without AOP:

- Repeated code occurs.
- Maintenance becomes difficult.

AOP solves these problems by:

- Separating common functionalities.

Important Terms in AOP

Term	Meaning
Aspect	Cross-cutting concern
Advice	Action performed
Join Point	Point of execution
Pointcut	Expression selecting join points

Advantages of AOP

1. Reduces code duplication.
2. Improves modularity.
3. Simplifies maintenance.
4. Better separation of concerns.

Example of AOP

```
@Before("execution(* add(..))")

public void log() {

    System.out.println("Logging");
}
```

Spring MVC Architecture

Spring MVC is web framework based on:

Model View Controller
architecture.

Spring MVC separates application into:

1. Model
2. View
3. Controller

Components of Spring MVC

Component	Purpose
Model	Business data
View	User interface
Controller	Request handling

Spring MVC Architecture Diagram

User
↓
DispatcherServlet
↓
Controller
↓
Service
↓
DAO
↓
Database

Advantages of Spring MVC

1. Separation of concerns.
2. Easy maintenance.
3. Better scalability.
4. Modular architecture.

Bean Factory

BeanFactory is core container of Spring Framework.

It manages:

- Bean creation
- Dependency injection

- Bean lifecycle

Features of BeanFactory

1. Lightweight container.
2. Lazy initialization.
3. Basic dependency injection support.

Example of BeanFactory

```
BeanFactory factory =
new XmlBeanFactory(
new FileSystemResource(
"beans.xml"));
```

Application Context

ApplicationContext is advanced container of Spring.

It extends BeanFactory and provides:

- Event handling
- Internationalization
- Automatic bean initialization

Advantages of ApplicationContext

1. Supports events.
2. Better enterprise support.
3. Automatic bean loading.

Example of ApplicationContext

```
ApplicationContext context =
new ClassPathXmlApplicationContext(
"beans.xml");
```

Difference Between BeanFactory and ApplicationContext

BeanFactory	ApplicationContext
Basic container	Advanced container
Lazy loading	Eager loading
No event support	Event support available

Beans in Spring

Beans are Java objects managed by Spring container.

Spring creates:

- Initializes

- Configures
- Manages

beans automatically.

Bean Configuration Example

```
<bean id="student"
class="Student"/>
```

Attaching and Populating Beans

Attaching beans means configuring dependencies between beans.

Populating beans means assigning values to bean properties.

Example

```
<bean id="student"
class="Student">

<property name="name"
value="Veer"/>

</bean>
```

Injecting Data Through Setters

Example

```
class Student {

   private String name;

   public void setName(String name) {

      this.name = name;
   }
}
```

XML Configuration

```
<property name="name"
value="Veer"/>
```

Injecting Data Through Constructors

Example

```
class Student {

   String name;

   Student(String name) {

      this.name = name;
   }
}
```

XML Configuration

```
<constructor-arg
value="Veer"/>
```

Event Handling in Spring: Spring supports event-driven programming.

Events are notifications generated during application execution.

Listening on Events

Event listeners respond to events generated by application.

Example of Event Listener

```
public class MyListener
implements ApplicationListener {

    public void onApplicationEvent(
    ApplicationEvent e) {

        System.out.println(
        "Event Occurred");
    }
}
```

Publishing Events

Spring applications can publish custom events.

Example of Publishing Event

```
context.publishEvent(
new ApplicationEvent(this));
```

Spring MVC Layering

Spring MVC uses layered architecture.

Layers of Spring MVC

1. Presentation Layer
2. Controller Layer
3. Service Layer
4. DAO Layer
5. Database Layer

1. Presentation Layer

Contains:

- JSP pages
- HTML pages

Responsible for:

- User interaction

2. Controller Layer

Handles:

- User requests
- Request routing

3. Service Layer

Contains:

- Business logic
- Validation

4. DAO Layer

DAO means:

Data Access Object
Responsible for:

- Database operations

5. Database Layer: Stores application data.

Dispatcher Servlet

DispatcherServlet is front controller of Spring MVC.

It handles all incoming requests.

Responsibilities of DispatcherServlet

1. Receives client requests.
2. Sends request to controller.
3. Processes response.
4. Returns view to user.

Dispatcher Servlet Flow

Client
↓
DispatcherServlet
↓
Controller
↓
View Resolver
↓
View

Writing a Controller

Controller handles client requests and returns response.

Example of Controller

```
@Controller
public class HomeController {
    @RequestMapping("/home")
    public String home() {
        return "home";
    }
}
```

Output

home.jsp page displayed

DAO (Data Access Object)

DAO layer handles database operations.

Responsibilities:

- Insert data
- Update data
- Delete data
- Retrieve data

Example of DAO

```
public class StudentDAO {
    public void save() {
        System.out.println(
        "Data Saved");
    }
}
```

Models in Spring MVC

Model represents application data.

Model transfers data between:

- Controller
- View

Example of Model

```
model.addAttribute(
"name","Veer");
```

Services in Spring MVC

Service layer contains business logic.

Responsibilities:

- Validation
- Data processing
- Business rules

Example of Service

```
@Service

public class StudentService {

    public void saveStudent() {

        System.out.println(
        "Student Saved");
    }
}
```

Spring Configuration File

Spring configuration file contains bean definitions and dependency configurations.

Usually:

applicationContext.xml

Example of Configuration File

```
<beans>

<bean id="student"
class="Student">

<property name="name"
value="Veer"/>

</bean>

</beans>
```

Error Handling Strategy in Spring

Error handling manages application exceptions properly.

Spring provides:

- Exception handling
- Validation support

- Error pages

Advantages of Error Handling

1. Prevents application crash.
2. Improves reliability.
3. Improves user experience.

Exception Handling Example

```
try {
    int x = 10/0;
}
catch(Exception e) {
    System.out.println(
    "Error Occurred");
}
```

Output

Error Occurred

Spring MVC Exception Handling

Example:

```
@ExceptionHandler(Exception.class)
public String error() {
    return "error";
}
```

Advantages of Spring Framework

1. Lightweight architecture.
2. Loose coupling.
3. Easy testing.
4. Better modularity.
5. Simplifies enterprise development.
6. Supports MVC architecture.
7. Supports dependency injection.
8. Integrates easily with other technologies.

Applications of Spring Framework

Spring is widely used in:

- Enterprise web applications
- Banking systems
- Cloud applications
- REST APIs
- E-commerce websites
- Microservices architecture